The goal of this thesis is to study the formal semantics of SSA – rather than do this "all at once," we will instead take a page from the "nanopass" school of compiler development and study various intermediate languages L, along with the *relationships* between them.

That means that the first few chapters of this thesis are going to be a bit repetitive: for each language L that we introduce, after we've given some informal intuition and examples and situated it in our hierarchy of intermediate representations, we're going to need to give a formal description of the language so that we can make precise and prove our claims.

Usually, this means a variant of the following narrative:

- We give L a formal syntax – usually parametrized by abstract sets of operations and constants
- We'll then often formalize L's "expected" behaviour by equipping it with a simple *operational semantics* – typically, we will "build up" to a full semantics for L, by starting with a simple rewriting-based semantics and then adding features like *state* and *nondeterminism* in stages.
- Then, we define *type theory* for L, consisting of:
  - A *type system* – which defines the types a term can have and the contexts in which they are interpreted. Usually, this will be parametrized by an abstract set of *base types*.

    Often, we will study many different languages using the same type system.
  - A *typing relation* – this typically consists of *typing rules* which tell us which terms are well-typed in a given context.
  - An *equational theory* – this gives us a notion of which terms may be considered *equivalent* within a given context

At this point, we've got a formal specification for a programming language:

- How to tell whether a term is *grammatical* – like "$x * (y + z)$" – or nonsense – like "$C++$"
- A collection of ways we can *interpret* a given term (natural number, string, graph, tensor, neural network...) – that is, a set of *types* A, B, C
- A description of the *contexts* in which we can interpret term – and a relation which tells us in which contexts a given term can be interpreted in a given way.

  Concretely: we might be able to interpret $x + x$ as an integer in the context $x : \mathbb{Z}$, or as a string in the context x : `str` – but $x + x$ doesn't make sense for x : `POBox`.
- An *operational semantics* which describes an *abstract machine* and tells us how to *execute* our programs on it
- An *equational theory* which tells us when two terms are "the same" in a particular context – for example, we might have $x + y \equiv y + x$ in the context $x : \mathbb{Z}, y : \mathbb{Z}$, but not in the context x : `str`, y : `str`.

  This can let us reason about programs, but also *optimize* them – for example, we might rewrite

$$(x + 5, x + 5, x + 5) \qquad \longrightarrow \qquad \mathsf{let}\ y = x + 5; 3 * y$$

Once we *have* such a specification, however, the question remains as to whether it's any *good*:

- How do we know our equational theory is *correct* – especially in the presence of subtle *potential* "gotchas" like the following optimization, which introduces UB *if* `n == 0` is possible:

```
for(int i = 0; i < n; i++) {
  int t = k / n;
  acc += t * arr[i];
}
```

⟶

```
int t = k / n;
for(int i = 0; i < n; i++) {
  acc += t * arr[i];
}
```

- In fact – how do we even *define* two programs to be equivalent, in the presence of side-effects?

TODO 1: denotational semantics – do we pull the operational semantics *down*?

do we even want it at all –

- **it's not part of the MVP...**
- **but it really helps justify our denotational semantics as correct – otherwise completeness seems particularly abstract**

1. S-Expressions

TODO 2: This include should shift heading levels down by 1

2. Evaluating Arithmetic Expressions

We'll start by giving a formal account of the simplest practical language – that of *first-order* arithmetic expressions, like $x + y$ and $a + (5 \cdot b) + \sin(3)$.

Rather than complicate our language by introducing binary operators (like $x + y$), unary operators (like $-x$), and n-ary function applications (like $f(x, y, z)$), we can instead normalize everything to *S-expressions* – or *sexprs* – of the form $(f\ e_1\ \ldots\ e_n)$. We can then uniformly write:

- binary operations like $x + y$ as binary sexprs $(+\ x\ y)$,
- unary operations $-x$ as unary sexprs $(-\ x)$,
- function appplications $f(x, y, z)$ as n-ary sexprs $(f\ x\ y\ z)$.

In particular, the *operator* f always comes first, and there is no order-of-operations – all bracketing is explicit. Restricting ourselves to *first-order* sexprs means that we always require operators f to be atomic symbols drawn from a fixed set O – in particular, we don't support *partial application*, like $((\mathsf{add}\ 2)\ 3)$, since that would require $f = (\mathsf{add}\ 2)$ to be a compound expression.

Formally, we define our *syntax* as follows:

Definition 2.1 (First-order S-expressions): fixing
- a set of *variables* $x \in N$
- a set of *operations* $f \in O$
- a set of *constants* $c \in V$

we define the set of *first-order S-expressions* $e \in \mathsf{STm}_N(O)$ to be given by the grammar

$$e ::= x \mid c \mid (f\ e^*)$$

We say a sexpr is *closed* if it contains no free variables – otherwise, it is *open* – where the *free variables* $\mathsf{use}(e)$ of a sexpr e are defined as follows:

$$\mathsf{use}(x) = \{x\} \qquad \mathsf{use}(c) = \{\} \qquad \mathsf{use}((f\ e_1\ \ldots\ e_n)) = \bigcup_i \mathsf{use}(e_i)$$

We'll write the set of closed sexprs as $\mathsf{STm}_\emptyset(O)$.

It's easy enough, in the language of simple arithmetic, to define what a closed sexpr "means", since we can just *evaluate* it to find out.

For example, we might have been taught to evaluate expressions from the "inside out", like so:

$$(+\ 5\ (\cdot\ 3\ 7)) = (+\ 5\ 21) = 26$$

We can formalize this intuition in terms of a *small-step operational semantics*: given, for each operation $f \in O$, a (partial) function $\mathsf{ev}(f) : V^* \to V$ telling us what value it returns when called with arguments $c_1, \ldots, c_n$, we may define a *reduction relation* $e \to e'$ on sexprs as follows:

$$\text{step}\ \frac{\mathsf{ev}(f)(c_1, \ldots, c_n) = c}{(f\ c_1\ \ldots\ c_n) \to c} \qquad \text{arg}\ \frac{c_1, \ldots, c_n \in V \qquad e_1 \to e_1'}{(f\ c_1\ \ldots\ c_n\ e_1\ \ldots\ e_m) \to (f\ c_1\ \ldots\ c_n\ e_1'\ \ldots\ e_m)}$$

Partiality here means that we don't need to assign a meaning to ill-defined expressions like $(/\ 1\ 0)$ or $(+)$ (when forced, 0 is a decent choice for both of these) – we can simply leave them as undefined.

Notice that we've fixed the *evaluation order* of sexprs to be left-to-right – that is; we have to finish evaluating all terms to the *left* of a given term before we can get to reducing it.

We say a term e *evaluates* to a constant c – written $e \Downarrow c$ – if there exists a finite sequence of reductions

$$e \to e_1 \to ... \to e_n \to c$$

i.e., if $e \to^* c$ – where R^* denotes the *reflexive, transitive closure* of a relation R.

We call a relation $e \Downarrow c$ a *big-step operational semantics* – as our original intuition for evaluation would suggest, c is a pretty good candidate for the "meaning" of a program $e \Downarrow c$.

In particular, we may define a partial function $\mathsf{ev}(e)$ on closed terms by induction as follows:

$$\mathsf{ev}(c) = c \qquad\qquad \mathsf{ev}((f\ e_1\ ...\ e_n)) = \mathsf{ev}(f)(\mathsf{ev}(e_1), ..., \mathsf{ev}(e_n))$$

It is straightforward to show that $e \Downarrow \mathsf{ev}(e)$ if and only if $\mathsf{ev}(e)$ is defined.

TODO 3: Consider alternate example: $(\cdot\ (+\ 1\ 1)\ x)$

- **Reduces to** $(\cdot\ 2\ x)$ **– so these should be "the same"**
- **Intuitively the same as** $(+\ x\ x)$
- **Intuitively different from** $(\cdot\ 3\ x)$

We still, however, don't know what an *open* term means: if we try to evaluate

$$(+\ (+\ 3\ 2)\ (\cdot\ x\ 0)) \to (+\ 5\ (\cdot\ x\ 0)) \to\ ???$$

the first time we encounter a variable, the relation defined above becomes *stuck* – that is, we reach a term e which is not a constant, and yet does not reduce to anything – so, according to our current conventions, is ill-defined like $(/\ 1\ 0)$.

If we fire up a Python REPL and try it, it seems to agree:

```
>>> 5 + (x * 0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    5 + (x * 0)
        ^
NameError: name 'x' is not defined
```

On the other hand, if x is in context – say $x = 7$ – this evaluates to 5 as we'd expect:

```
>>> x = 7
>>> 5 + (0 * x)
5
```

What this hints at is that the operational semantics of an open term e depends on the context in which it is evaluated – here, on the values of its free variables.

We can model this by defining a *contextual reduction relation* $\gamma \vdash e \to e'$ parametrized by an *evaulation context* γ – a finitely supported function $\gamma : N \rightharpoonup V$ assigning a values to variables – as follows:

$$\text{var}\,\frac{\gamma(x) = c}{\gamma \vdash x \to c} \qquad \text{step}\,\frac{\mathsf{ev}(f)(c_1, ..., c_n) = c}{\gamma \vdash (f\ c_1\ ...\ c_n) \to c}$$

$$\text{arg}\,\frac{c_1, ..., c_n \in V \qquad \gamma \vdash e_1 \to e_1'}{\gamma \vdash (f\ c_1\ ...\ c_n\ e_1\ ...\ e_m) \to (f\ c_1\ ...\ c_n\ e_1'\ ...\ e_m)}$$

We can then evaluate

$$(x \mapsto 7) \vdash (+\ (+\ 3\ 2)\ (\cdot\ x\ 0)) \to (+\ 5\ (\cdot\ x\ 0)) \to (+\ 5\ (\cdot\ 7\ 0)) \to (+\ 5\ 0) \to 5$$

TODO 4: substitution maps open terms to closed terms

TODO 5: discuss:
- **algebra: "equivalence under substitution"**
- **programming: "observational equivalence"**

Given a set of *types* $A \in \mathcal{T}$, we'll define a *stack-typing relation* on O to be any relation

$$f : [A_1, ..., A_m] \to [B_1, ..., B_n]$$

where

- $A_1, ..., A_m \in \mathcal{T}$ are types – m is called the *input arity* or *arity* of f.
- $B_1, ..., B_n \in \mathcal{T}$ are types – n is called the *output arity* of f.

For now, we'll only consider operations with an output arity of 1.

We define a *(typing) context* $\Gamma \in \mathsf{Ctx}_N(\mathcal{T})$ to be given by a list of *bindings*

$$x_1 : A_1, ..., x_n : A_n$$

We can define a simple type system for S-expressions by giving a typing relation $\Gamma \vdash e : A$ that says "under the assumptions of Γ, the S-expression e has type A". The rules are:

TODO 6: Typing rules for constants – as distinct from nullary ops

$$\text{var-head}\,\frac{}{\Gamma, x : A \vdash x : A} \qquad \text{var-tail}\,\frac{\Gamma \vdash y : B,\ x \neq y}{\Gamma, x : A \vdash y : B}$$

$$\text{app}\,\frac{f : [A_1, ..., A_n] \to [B] \qquad \forall i. \Gamma \vdash a_i : A_i}{\Gamma \vdash (f\ a_1\ ...\ a_n) : B}$$

TODO 7: *Soundness* of typing: preservation of typing

TODO 8: *Completeness* of typing: in the presence of *simple types*

TODO 9: Church vs. Curry – we will only consider *well-typed* terms

TODO 10: Doesn't lose generality – unityping works in this framework due to multi-arity

TODO 11: *Substitution* and *observational equivalence* – typing rules for substitution – two open programs are equivalent iff equivalent under every substitution – this is usually where *metavariables* come in

TODO 12: *Purity* – required for substitution to preserve equivalence – typing rules for purity

TODO 13: Denotational semantics of S-expressions: partial functions

TODO 14: Denotational semantics of S-expressions: *soundness* and *completeness*

TODO 15: Equational theory of S-expressions: *purity* – as otherwise you can't *introduce* divisions

TODO 16: Equational theory of S-expressions: *soundness* and *completeness*

3. State

TODO 17: Operational semantics of S-expressions – *state*

TODO 18: *Observational equivalence* in the presence of state

TODO 19: Denotational semantics of S-expressions – the partial state monad

TODO 20: Denotational semantics of S-expressions: *soundness* and *completeness*

TODO 21: Equational theory of S-expressions: *purity*

TODO 22: Equational theory of S-expressions: *soundness* and *completeness*

4. Sequencing

5. Branching

6. Loops

7. Unstructured Control-Flow